

Conditional Statements (Conditionals): if, elif, else

Contributions

Dania: ex. 4

Neha: ex. 2

Rohit: ex. 3

We collaborated on ex. 1, then split the work and reviewed it all together in the end.

Conditionals are used to check whether a condition is True or False and allow the program to react accordingly. The simplest form is the if statement.

In []:

In [46]:

```
cat_count = 4

if cat_count == 0:
    print('Mice are safe!')
```

The if, else statement works when you want one thing to happen when a condition it true, and something else to happen when it is false.

In [47]:

```
cat_count = 4

if cat_count == 0:
    print('Mice are safe!')
else:
    print('🐱🐱 Meow Meow')
```

🐱🐱 Meow Meow

Sometimes there are more than two possibilities. One way to express a decision tree like that is a chained conditional, using elif. Remember `if` always comes first, `elif` in the middle, and `else` is last.

In [48]:

```
cat_count = -2 # Try this with different numerical values

if cat_count == 0:
    print('Mice are safe!')
elif cat_count > 0:
    print('🐱🐱 Meow Meow')
else:
    print('How do you have a negative amount of cats?')
```

How do you have a negative amount of cats?

One conditional can also be nested within another. However, nested conditionals very quickly become difficult to read. In general, it is a good idea to avoid too many of them when you can.

```
In [49]: x=15#initial value

if x<20: #if statement
    print('Value is:', x)

print('\n')
correct_value = True
if type(x) is int:
    print('Acceptable value is:', x)
else:
    print("Value of x is unacceptable")
    correct_value = False

print('\n') # newline character prints new lines
if correct_value:
    if x>20: #chained conditional
        print("Value %s is greater than 20"%x)
    elif x<10:
        print("Values %s is less than 10"%x)
    else:
        print('The value', x, 'is between 10 and 20')
```

Value is: 15

Acceptable value is: 15

The value 15 is between 10 and 20

try, except

Try, except statements are useful for when something goes wrong.

```
In [50]: cat_count = 'dog' # Something is wrong here

try:

    if cat_count == 0:
        print('Mice are safe!')
    elif cat_count > 0:
        print('😸😸 Meow Meow')
    else:
        print('How do you have a negative amount of cats?')

except Exception as e:

    print(e)
```

'>' not supported between instances of 'str' and 'int'

Iterations

Computers can automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well.

There are two (main) types of loops. The first uses a `for` statement to iterate over a defined amount of loops. The second, a `while` loop, iterates until a condition is met. While loops can iterate indefinitely if not used correctly. -Be careful!!!

For Loops

```
In [2]: # Use the range function to print out a number of specific n

loop_count=0

for i in range(5):
    print(i) #reports the current number of iterations
    loop_count +=1 # increase the value of the loop counter by 1

print(f'The number of loops is: {loop_count}')
```

```
0
1
2
3
4
The number of loops is: 5
```

```
In [3]: range(5)
```

```
Out[3]: range(0, 5)
```

```
In [4]: for number in range(3, 6):
        print(number)
```

```
3
4
5
```

```
In [5]: for index in range(2, 10): #Notice where in index starts and stops
        print(index)
```

```
2
3
4
5
6
7
8
9
```

```
In [6]: # Populating a dictionary through a for loop
dct1 = {}

for index in range(1, 10):
    dct1[index] = f'The value is {index}'

print(dct1)
```

```
{1: 'The value is 1', 2: 'The value is 2', 3: 'The value is 3', 4: 'The value is 4', 5: 'The value is 5', 6: 'The value is 6', 7: 'The value is 7', 8: 'The value is 8', 9: 'The value is 9'}
```

```
In [7]: dct1.keys()
```

```
Out[7]: dict_keys([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [8]: dct1.values()
```

```
Out[8]: dict_values(['The value is 1', 'The value is 2', 'The value is 3', 'The value is 4', 'The value is 5', 'The value is 6', 'The value is 7', 'The value is 8', 'The value is 9'])
```

```
In [ ]: #for loop demo
#general case: for loop_variable in sequence:
x=1
y=2
z=3

list_one= [x,y,z]

loop_count=1

for list_item in list_one: #this for loop is going to iterate through the elements of list_one
    print("Loop # %s \nItem %s"%(loop_count, list_item))
    loop_count=loop_count+1

print('\n')

#using for with range()

for i in range(10): #this is loops through a range of 0-9 (10 items), and prints each item
    print(i)
```

```

Loop # 1
Item 1
Loop # 2
Item 2
Loop # 3
Item 3

```

```

0
1
2
3
4
5
6
7
8
9

```

While Loops

```

In [ ]: count = 0
        while (count < 9):
            print('The count is:', count)
            count += 1

```

```

In [ ]: conditional = True
        x = 0
        while conditional:
            print(f"x = {x}")
            if x>=30:
                conditional = False
            x+=3

```

```

In [ ]: conditional = True
        x = 0
        while conditional:
            print(f"x = {x}")
            if x>=30:
                conditional = False
            x+=3

```

```

In [ ]: # this loop would run forever, but uses a break statement when a condition i
        y = 0
        while True:
            print(f"y = {y}")
            if y>=20:
                break # ends the loop
            y+=2

        # Be careful to avoid infinite loops as they can crash your computer!!!

```

```

In [ ]: #while loop demo
        #while loops are the second basic loop type

```

```
# these loops run until a condition is met

print("\nEx 1\n")
#This loop counts from 0 - 9 like "for i in range(10)"
count = 0
while (count < 9):
    print('The count is:', count)
    count = count + 1

print("\nEx 2\n")
#this loop uses a Boolean variable to end the loop when a condition is met
conditional = True
x = 0
while conditional:
    print("x = %s"%x)
    if x>=30:
        conditional = False
    x+=3

print("\nEx 3\n")
# this loop would run forever, but uses a break statement when a condition is met
y = 0
while True:
    print("y = %s"%y)
    if y>=20:
        break
    y+=2
```

Lab 2- Clinic Flow Simulator — Control Flow Lab

 **Skills:** dictionaries, if/elif/else, for loops, while loops, tuples, try/except, debugging with print()

Story setup

You are shadowing in a busy Emergency Department (ED). The team is short-staffed and the waiting room is full. Your job is to help the nurse team by writing small pieces of Python that support the workflow.

Important: This is a **fictional** scenario. All thresholds and rules are made up for learning programming.

How this lab works

- Each step adds new information to the same `patient` record.
- Each step **feeds into the next**.

Excercise 1 — Intake: Ava arrives at the Emergency Department (Dictionary)

Ava walks into the ED with **feverish chills** and says she has felt worse over the last few hours. A triage nurse takes quick vitals.

Your task

Build a dictionary called `patient` that stores:

- name
- severity (0–10)
- heart rate
- temperature in Celsius
- print out the dictionary you created

```
In [51]: print("=== Step 1: Intake ===")

# Ava's intake data

patient_name = "Ava"
severity_value = 6      # 0–10 (how bad she feels)
heart_rate_value = 92   # beats per minute
temp_c_value = 38.1     # degrees C

patient = {}
```

=== Step 1: Intake ===

```
In [53]: #Answer Here: Store the intake data into the patient dictionary
patient = {'patient_name': patient_name, 'severity_value': severity_value, '
```

Exercise 2 — Triage: The Emergency Department is overloaded (if/elif/else)

The waiting room is packed. The charge nurse needs a quick triage label so the team knows who should be seen first. You are using a fictional triage rule:

Triage rules

- **RED** if severity ≥ 8 OR hr > 130 OR temp ≥ 40
- **YELLOW** if severity ≥ 5
- **GREEN** otherwise

Your task

Write `try/except` logic to search for missing intake information (severity, hr, temp).
Print patient information.

Write `if/elif/else` logic to compute `triage` and store it in `patient["triage"]`.

Add print statements to show which rule triggered and the updated patient directory.

```
In [55]: print("\n=== Step 2: Triage ===")

# Pull values from the dict.
#Answer Here: Write try/except blocks to get severity, heart rate, and temperature

#Answer Here: Write triage logic.
# HINT: check RED first (highest priority)

# Pull values from the dict.
#Answer Here: Write try/except blocks to get severity, heart rate, and temperature

# severity = patient[severity_value]
# hr = patient[heart_rate_value]
# temp = patient[temp_c_value]

# try:
#     if patient[patient_name] is not None:
#         name = patient[patient_name]
#     if severity is None:
#         print('sev missing')
#     if hr is None:
#         print('hr missing')
#     if temp is None:
#         print('temp missing')

# except Exception as e:
#     print(e)

if patient['severity_value'] >= 8 or patient['heart_rate_value'] > 30 or patient['temp_c_value'] > 40:
    patient["triage"] = "RED"
    print("Rule triggered: RED")
elif patient['severity_value'] >= 5:
    patient["triage"] = "YELLOW"
    print("Rule triggered: YELLOW")
else:
    patient["triage"] = "GREEN"
    print("Rule triggered: GREEN")

print(patient)
```

=== Step 2: Triage ===

Rule triggered: RED

```
{'patient_name': 'Ava', 'severity_value': 6, 'heart_rate_value': 92, 'temp_c_value': 38.1, 'triage': 'RED'}
```

Excerise 3 — Orders: What do we do first? (Nested if)

A nurse asks Ava *what her main symptom is right now*.

Patient order rules

- cough → "Viral panel + hydration"
- rash → "Allergy review + topical test"
- pain → if severity ≥ 7 then "Imaging" else "Rest protocol"
- other → "General assessment"

Your task

Use `if/elif/else` (with a nested if for pain) to pick an order

Store it in `patient["order"]`

Print the order AND print the updated dictionary

```
In [44]: print("=== Step 3: Orders ===")
symptom_raw = "pain"
print("Patient symptom:", (symptom_raw))

sev = patient["severity_value"]
print("Severity used for orders:", sev)

# Answer Here: choose a care order for Ava using if/elif/else logic
if symptom_raw == 'pain':
    if severity_value >= 7:
        order = 'imaging'
    else:
        order = 'Rest protocol'
elif symptom_raw == 'cough':
    order = 'Viral panel + hydration'
elif symptom_raw == 'rash':
    order = 'Allergy review + topical test'
else:
    order = 'General assessment'

if order is not None:
    patient["order"] = order #Updates patient dict with order

#print Order
print(order)
#print patient dict
print(patient)
```

```
=== Step 3: Orders ===  
Patient symptom: pain  
Severity used for orders: 6  
Rest protocol  
{'patient_name': 'Ava', 'severity_value': 6, 'heart_rate_value': 92, 'temp_c_value': 38.1, 'order': 'Rest protocol', 'triage': 'RED'}
```

Exercise 4 — Tracking: The board needs a patient list (List of dicts)

The charge nurse keeps a running “board” of patients who are waiting. You will add Ava to the board.

Your task

Create a list named `patients` and append Ava's `patient` dict. Print the newly created list.

```
In [45]: print("=== Step 4: Patient Board ===")
```

```
patients = []
```

```
#Answer Here
```

```
patients.append(patient)
```

```
print('Patients:', patients)
```

```
=== Step 4: Patient Board ===  
Patients: [{'patient_name': 'Ava', 'severity_value': 6, 'heart_rate_value': 92, 'temp_c_value': 38.1, 'order': 'Rest protocol', 'triage': 'RED'}]
```